Mejoras de índole técnica

- **Mejorando la mutación y cruce.**
- **Inercia o pseudo gradientes.**
- **Utilizar la GPU (unidad de procesamiento gráfico).**

Mejorando la mutación y cruce.

La implementación en la DLL del algoritmo genético realiza la mutación y cruce según lo descrito en el libro:

1. Mutación: Se crea una población de individuos mutantes, a partir del individuo de referencia, aplicando la fórmula 18. El uso del 'Mejor individuo' es lo que determina la denominación '**best**' en 'DE/**best**/1/bin'. La inclusión del '**1**' en la denominación 'DE/best/**1**/bin' hace referencia a que sólo hay una resta. Como puede verse, el individuo mutante en la posición 'n' de la población es totalmente independiente del individuo original en esa misma posición, la relación con el individuo original se produce en la recombinación. En este punto, la nueva población está compuesta por mutantes del “Mejor individuo” que actuará como primer progenitor de todos los hijos que surgirán en el paso siguiente: la recombinación (una práctica habitual en la naturaleza).

2. Recombinación o cruce: A continuación, se procede a la recombinación (crossover) de los individuos mutantes con los de la población original (que podemos considerar que actúan como segundo progenitor). Para cada individuo ‘n’ de la población, se construye un vector de prueba (hijo) rellenándolo secuencialmente con valores del vector 'Mutante' (primer progenitor) o del vector del individuo original (segundo progenitor). La decisión de usar el valor del 'Mutante', o del individuo original, se toma mediante una distribución binomial (de aquí el término '**bin**' en 'DE/best/1/**bin**'), para ello, se genera un número aleatorio en [0, 1), si este número es menor que un determinado parámetro, que se denomina constante de recombinación, el parámetro se toma del 'Mutante', de lo contrario, se toma desde el individuo original. Se exige que siempre se incluya, como mínimo, un valor del vector 'Mutante' (seleccionado aleatoriamente), para evitar que la nueva solución a probar sea una copia exacta de la original. En todos los casos que hemos estudiado ha resultado más eficiente tomar siempre el parámetro del mutante, lo cual ocurre si la constante recombinación es 1.

$$Mutante = Mejor\ individuo + mutación * (Individuo_{azar\ 1} - Individuo_{azar\ 2})$$

Fórmula 18. Generación de un individuo mutante mediante la estrategia "DE/best/1/bin"

Este procedimiento que hemos llamado "DE/best/1/bin", en SCIPY es el "rand1". SCIPY denomina "best1bin" a la que estrategia para la mutación que utiliza el mejor individuo y solo un individuo de la población al azar y no dos (fórmula 19).

$$Mutante = Mejor\ individuo + mutación * (Mejor\ individuo - Individuo_{azar\ 1})$$

Fórmula 19. Generación de un individuo mutante mediante la estrategia "DE/best/1/bin"

La estrategia de la fórmula 19 no converge en absoluto, colapsa rápidamente sobre el mejor individuo de la primera generación, por lo que no la hemos utilizado.

Otra variante habitual es que el mutante no se obtenga añadiendo al mejor individuo la diferencia de dos individuos generados al azar, en su lugar, esta diferencia se añade a un tercer individuo también elegido al azar. Esta estrategia converge, pero mucho más lentamente que la fórmula 18. Conceptualmente, no es una búsqueda Monte Carlo de paso variable, porque no partimos de la mejor solución de una iteración a la siguiente.

En la fórmula 18, tenemos una situación particular cuando los dos individuos elegidos al azar (i1 y i2) son el mismo, en este caso el individuo mutante tendrá los valores de las variables correspondientes al **mejor individuo de la generación anterior**, si el sorteo que se realiza al efecto es inferior a la constante de recombinación, y del correspondiente individuo[1], en caso contrario. Como el valor de la constante de recombinación que hemos utilizado es del 0,999, el 99,9% de las veces se tomará el valor del mejor individuo[2], es decir, el mutante será cuasi idéntico al mejor individuo de la generación anterior.

La probabilidad de que el sorteo de dos individuos conduzca al mismo individuo es la inversa del número de individuos en la población (por ejemplo, para el caso de 5400 individuos resulta del 0,00018 = 0,018%), lo que significa que solo ocurrirá una vez por generación (se generan un número de mutantes igual al de la población) y a pesar de esto, si descartamos el mutante cuando ambos individuos i1 y i2 son el mismo, la convergencia del algoritmo se ralentiza de

[1] En cada generación se realizan tantas mutaciones como individuos y por "correspondiente individuo" nos referimos al individuo en la misma posición que el número de mutación que se está realizando. Como se ve en la fórmula 18, la mutación "n" no guardan ninguna relación con el individuo en esa misma posición, pero si en el proceso de selección porque se compara el mérito de mutación e individuo, posición a posición.

[2] Si el correspondiente sorteo produce un valor superior a una constante de recombinación de 0,999 se toma el valor de la variable del individuo al azar, cosa que para 251 variables ocurre, en promedio, tan solo 0,24 veces.

forma importante, pasamos de necesitar 140644 generaciones a 254321, casi el doble. **Retener el mejor individuo de la generación anterior acorta significativamente el número de generaciones necesarias para encontrar la solución**.

A la vista de este resultado, era necesario ver que ocurre si en cada generación generamos más de un mutante cuasi idéntico al mejor individuo de la generación anterior.

El resultado es que la convergencia se alcanza dos o tres veces más rápido, es decir, el tiempo de cálculo se reduce a la mitad o a la tercera parte.

Los resultados de utilizar diversas probabilidades (PRM), de crear mutantes cuasi idénticos al mejor de la generación anterior, para un caso del modelo de 31 sectores, se muestran en la tabla1 y la figura 1. La probabilidad óptima es 3,467E-03 (0,35%) y se necesitan 2,3 veces menos generaciones que cuando PRM es 1.667E-04[3] que es lo que venía ocurriendo con la aplicación exacta de la fórmula 18.

[3] El PRM es 1,667E-4 porque se siguen creando mutantes, cuasi iguales al mejor, cuando los dos individuos elegidos al azar resultan ser el mismo, lo que en este caso tiene dicha probabilidad.

	1 / NPOB				1 / NPOB			
	1.852E-04				1.667E-04			
NPOB	**5400**				**6000**			
PRM add	**PRM total**	**N Extra**	**N gen**	**N VAB**	**PRM total**	**N Extra**	**N gen**	**N VAB**
	0.000E+00		283261	200000	0.000E+00	0.0	254321	180000
0.00E+00	1.852E-04	1.0	118488	90000	1.667E-04	1.0	140644	100000
1.00E-03	1.185E-03	6.4	81914	60000	1.167E-03	7.0	79724	60000
1.90E-03	2.085E-03	11.3	77733	60000	2.067E-03	12.4	57019	50000
2.10E-03	2.285E-03	12.3	76798	50000	2.267E-03	13.6	63449	50000
2.30E-03	2.485E-03	13.4	71246	50000	2.467E-03	14.8	60273	50000
2.50E-03	2.685E-03	14.5	67828	50000	2.667E-03	16.0	64554	50000
2.70E-03	2.885E-03	15.6	76015	50000	2.867E-03	17.2	58348	40000
3.00E-03	3.185E-03	17.2	72687	60000	3.167E-03	19.0	70780	50000
3.30E-03	3.485E-03	18.8	63544	50000	**3.467E-03**	**20.8**	54356	50000
3.50E-03	3.685E-03	19.9	66998	50000	3.667E-03	22.0	60624	50000
3.70E-03	3.885E-03	21.0	64460	50000	3.867E-03	23.2	58339	40000
4.00E-03	4.185E-03	22.6	63191	50000	4.167E-03	25.0	60859	50000
4.30E-03	4.485E-03	24.2	67287	50000	4.467E-03	26.8	58946	40000
5.00E-03	5.185E-03	28.0	63879	50000	5.167E-03	31.0	63032	50000
6.00E-03	6.185E-03	33.4	71747	60000	6.167E-03	37.0	62439	50000
8.00E-03	8.185E-03	44.2	61573	50000	8.167E-03	49.0	61528	50000
1.00E-02	1.019E-02	55.0	73478	60000	1.017E-02	60.9	58130	40000
1.10E-02	1.119E-02	60.4	86389	70000	1.117E-02	67.0	57868	40000
1.20E-02	1.219E-02	65.7	91312	80000	1.217E-02	73.0	67501	50000
1.40E-02	1.419E-02	76.6	68994	50000	1.417E-02	85.0	59150	50000
1.60E-02	1.619E-02		- NC -		1.617E-02	97.0	70220	60000
1.80E-02	1.819E-02		- NC -		1.817E-02	109.0	87670	70000
2.00E-02	2.019E-02		- NC -		2.017E-02	- NC -		

Tabla 1. Modelo de 31 sectores y dos valores para la población inicial: 5400 y 6000 individuos. Generaciones necesarias para alcanzar el objetivo de error, en función de la probabilidad (PRM total) de generar mutantes cuasi idénticos al mejor de la generación anterior. "N VAB" es la generación a partir de la cual ya no cambia el valor añadido bruto. "– NC-" significa que los cálculos convergen en un mínimo local inaceptable.

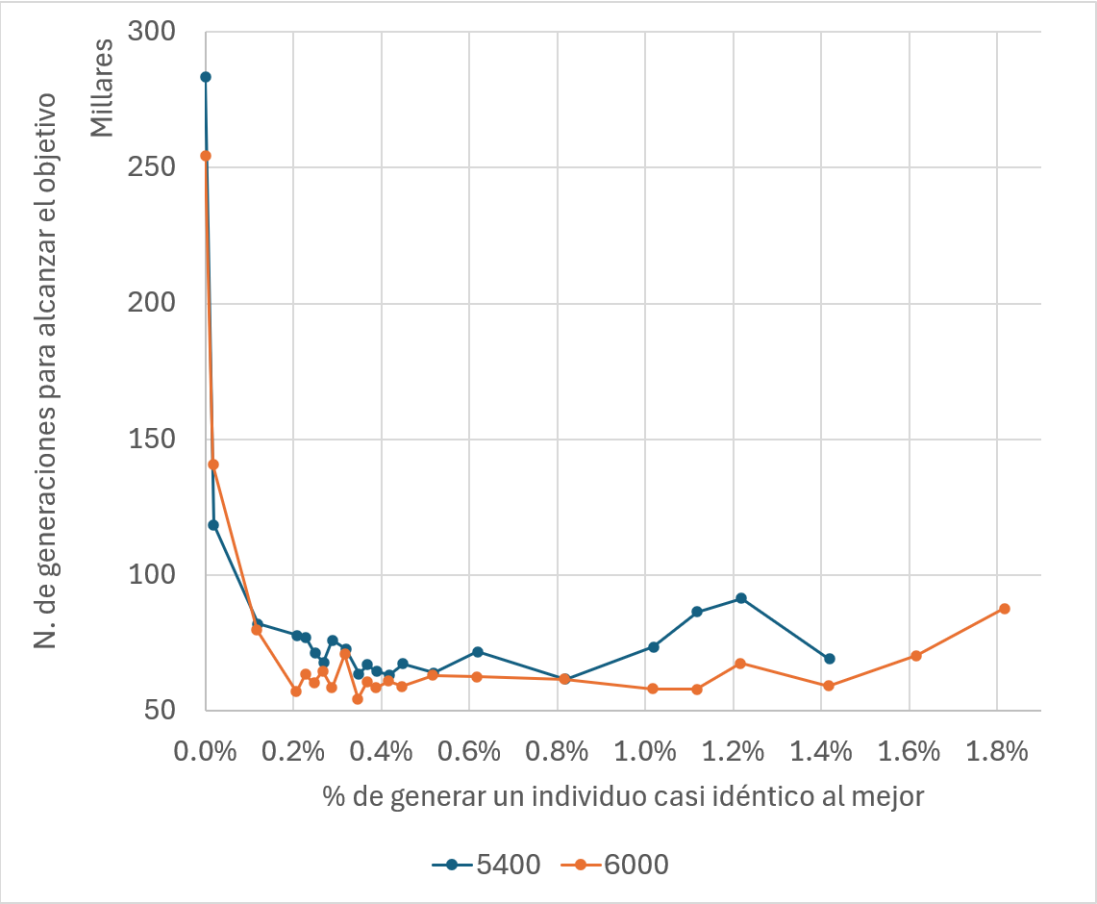


Figura 1. Modelo de 31 sectores. Número de generaciones necesarias para alcanzar el objetivo de error, en función de la probabilidad (PRM) de generar mutantes cuasi idénticos al mejor de la generación anterior.

Para el modelo de 63 sectores, el PRM óptimo es 8,452E-04, muy inferior al, 3,467E-03 del caso con 31 sectores, y el número de generaciones para alcanzar el objetivo pasa de pasa de 1500000 a 362879. Los resultados se muestran en la taba 2 y figura 2.

	NPOB	**1 / NPOB**	
	20010	5.00E-05	
PRM add	**PRM total**	**N gen**	**N VAB**
0.0E+00	9.524E-05	1500000	
3.0E-04	3.952E-04	808271	320000
6.0E-04	6.952E-04	386044	180000
7.5E-04	**8.452E-04**	**362879**	**140000**
8.1E-04	9.052E-04	380941	220000
1.0E-03	1.095E-03	402943	150000

Tabla 2. Modelo de 63 sectores. Generaciones necesarias para alcanzar el objetivo de error, en función de la probabilidad (PRM total) de generar mutantes cuasi idénticos al mejor de la generación anterior. "N VAB" es la generación a partir de la cual ya no cambia el valor añadido bruto

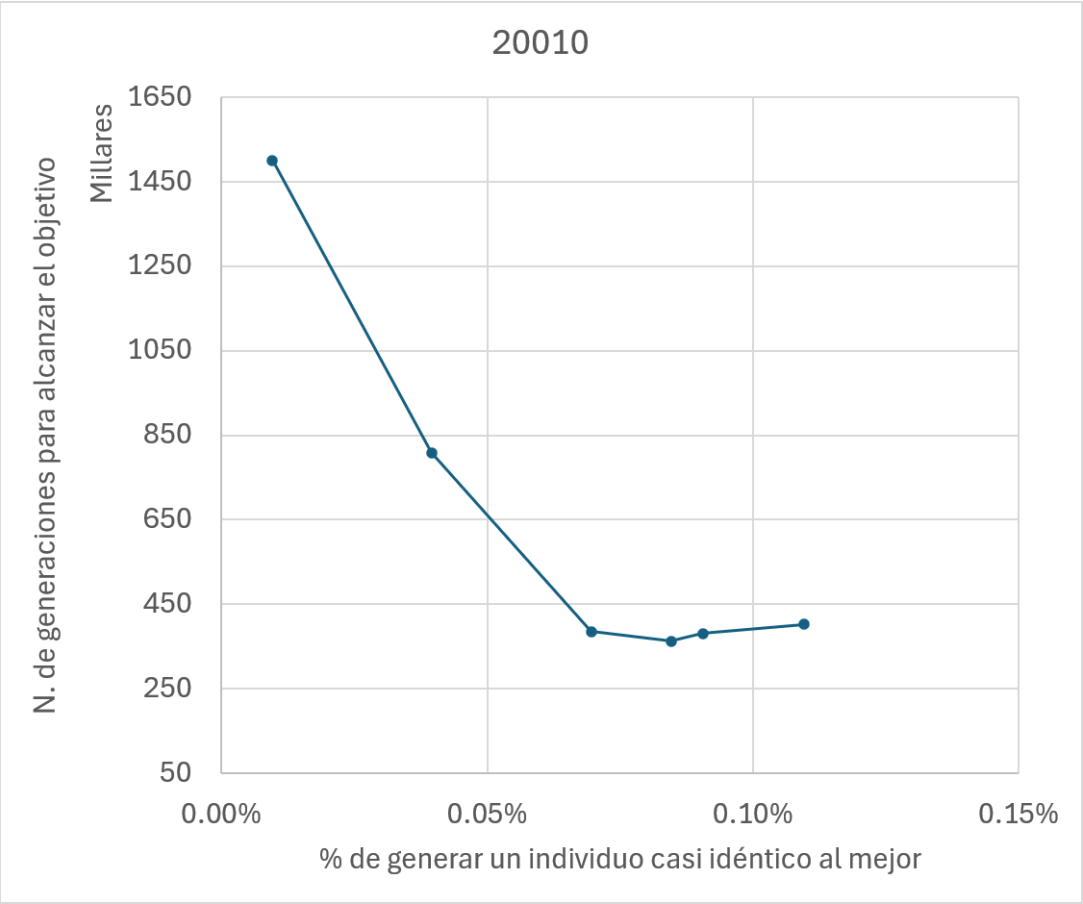


Figura 2. Modelo de 63 sectores. Número de generaciones necesarias para alcanzar el objetivo de error, en función de la probabilidad (PRM total) de generar mutantes cuasi idénticos al mejor de la generación anterior.

La figura 3 muestra la evolución de los cálculos con el modelo de 63 sectores para diferentes valores del PRM adicional al que, de por sí, existe cuando i1=i2 (4,998E-05).

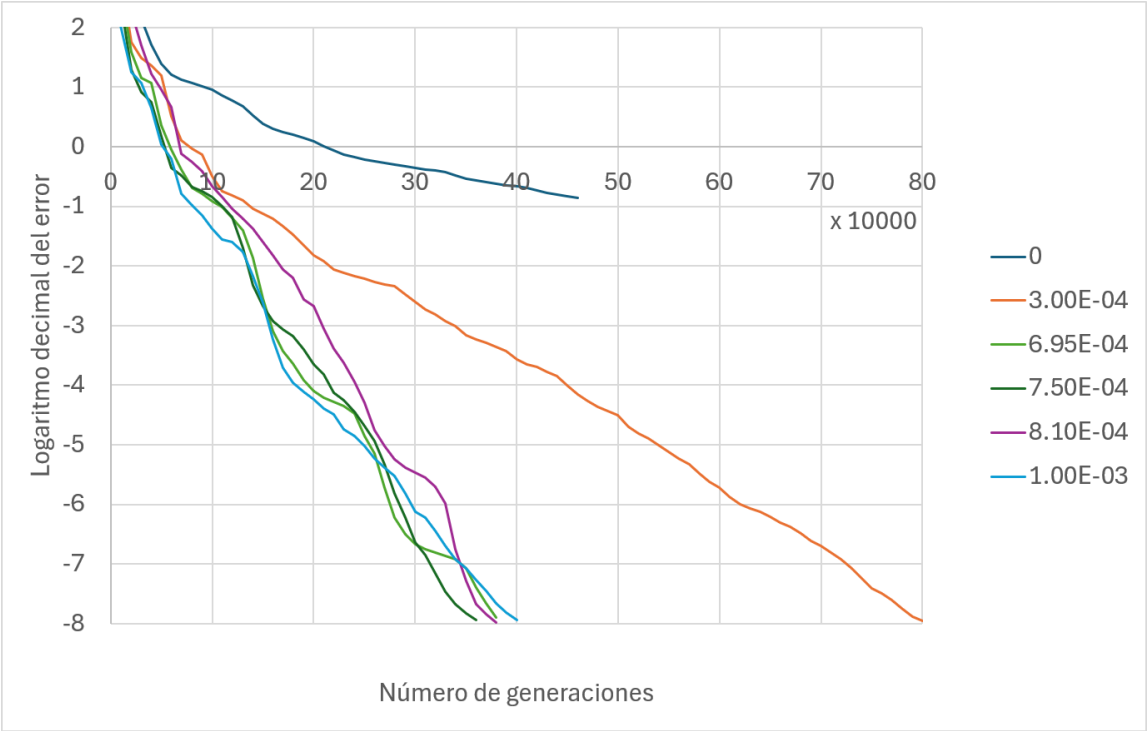


Figura 3. Modelo de 63 sectores. Evolución de los cálculos, en función de la probabilidad (PRM adicional) de generar mutantes cuasi idénticos al mejor de la generación anterior.

Hasta aquí nos hemos centrado en el número de mutantes cuasi iguales al mejor. Ahora analizaremos el número de variables que se toman del mejor individuo de la generación anterior para generar el mutante, esto depende directamente de la constante de recombinación, hasta ahora, la hemos fijado en 0.999, el mismo valor que la constante de recombinación general del algoritmo. En la tabla 1, bajo la columna “N extra”, se muestra el número de mutantes cuasi idénticos creados en cada generación, que para la PRM óptima es de 20,8.

Sin cambiar la constante de recombinación general (0.999), hemos ido variando la utilizada expresamente cuando se genera un mutante casi igual al mejor de la generación anterior (CRE) y los resultados, para el caso de 31 sectores y un PRM adicional de 3,30E-03, están en la tabla 3 y figura 4.

	PRM add	PRM total	N Extra
	3.30E-03	3.49E-03	20.8
NPOB		**6000**	
CRE	**N CRE**	**N gen**	**N VAB**
0.9800	4.84	50038	40000
0.9850	3.63	45538	40000
0.9900	2.42	47747	40000
0.9950	1.21	42084	30000
0.9990	0.24	62695	50000
0.9995	0.12	74519	60000
1.0000	0.00	79656	60000

Tabla 3. Resultados para el caso de 31 sectores y un PRM adicional de 3,30E-03, para diferentes valores de la constante de recombinación que se utiliza exclusivamente para los mutantes cuasi idénticos (CRE).

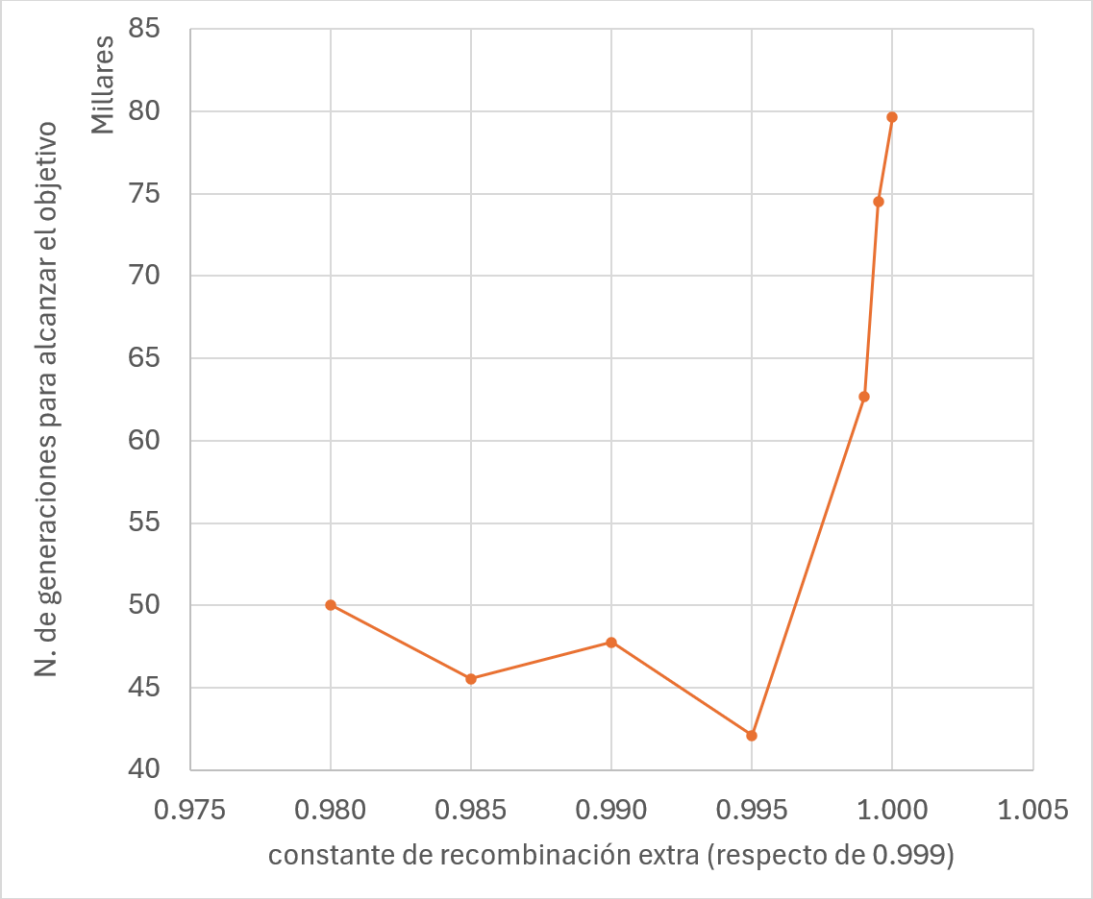


Figura 4. Resultados para el caso de 31 sectores y un PRM adicional de 3,30E-03, para diferentes valores de la constante de recombinación que se utiliza exclusivamente para los mutantes cuasi idénticos (CRE).

Aplicando el valor de 0,995 para la constante de recombinación, que se utiliza exclusivamente para los mutantes cuasi idénticos (CRE), a las filas de la tabla 1, el número de generaciones para alcanzar el objetivo de error se muestra en la figura 5, en comparación al que figura en la tabla 1.

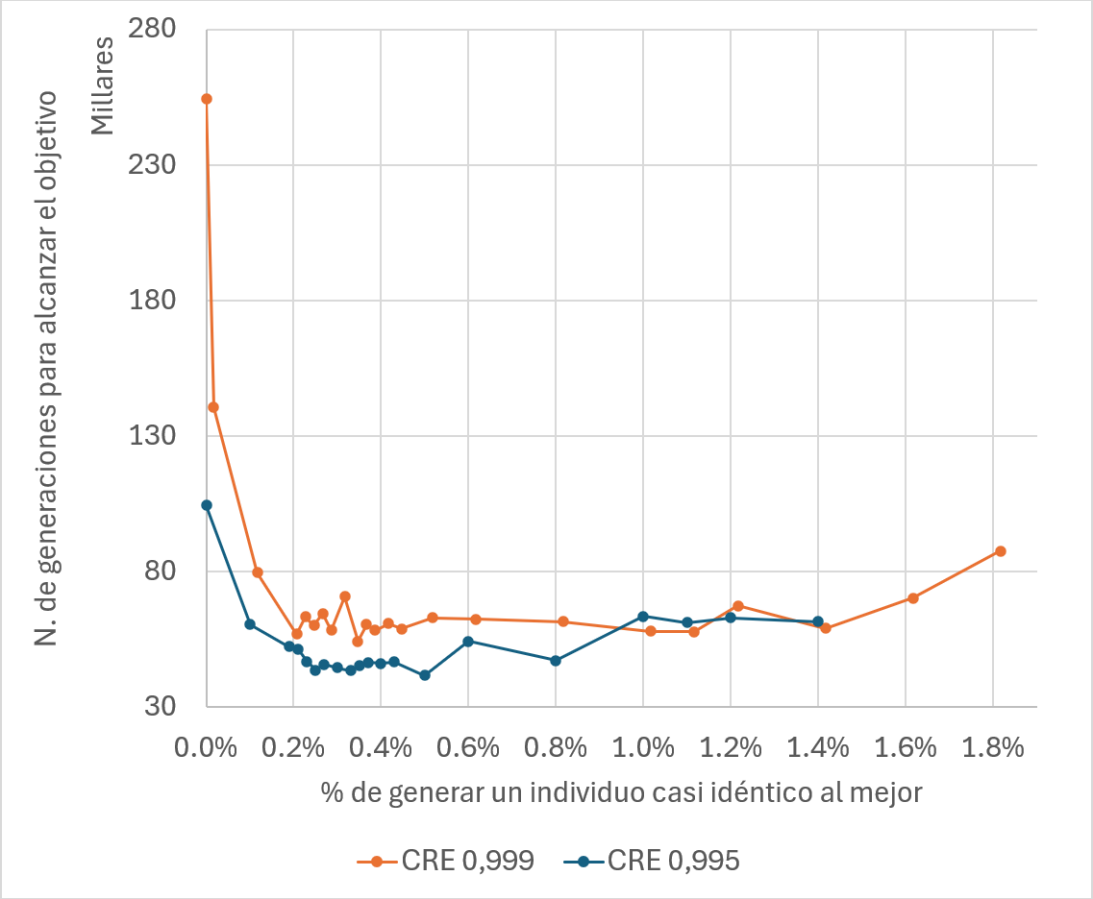


Figura 5. Número de generaciones para alcanzar el objetivo de error, aplicando el valor de 0,995 a la constante de recombinación exclusiva para los individuos cuasi idénticos (CRE), a las filas de la tabla 1, en comparación con el número de generaciones que figura en la tabla 1.

La figura 6 compara la evolución de los cálculos con el modelo de 63 sectores y una PRM adicional de 7,5E-03 (8.000E-04 de PRM total) cuando se usan como constante de recombinación (CRE) los valores: 0,9995, 0,9980, 0,9975 y 0,9970. Para un CRE de 0,9965 los cálculos no convergen.

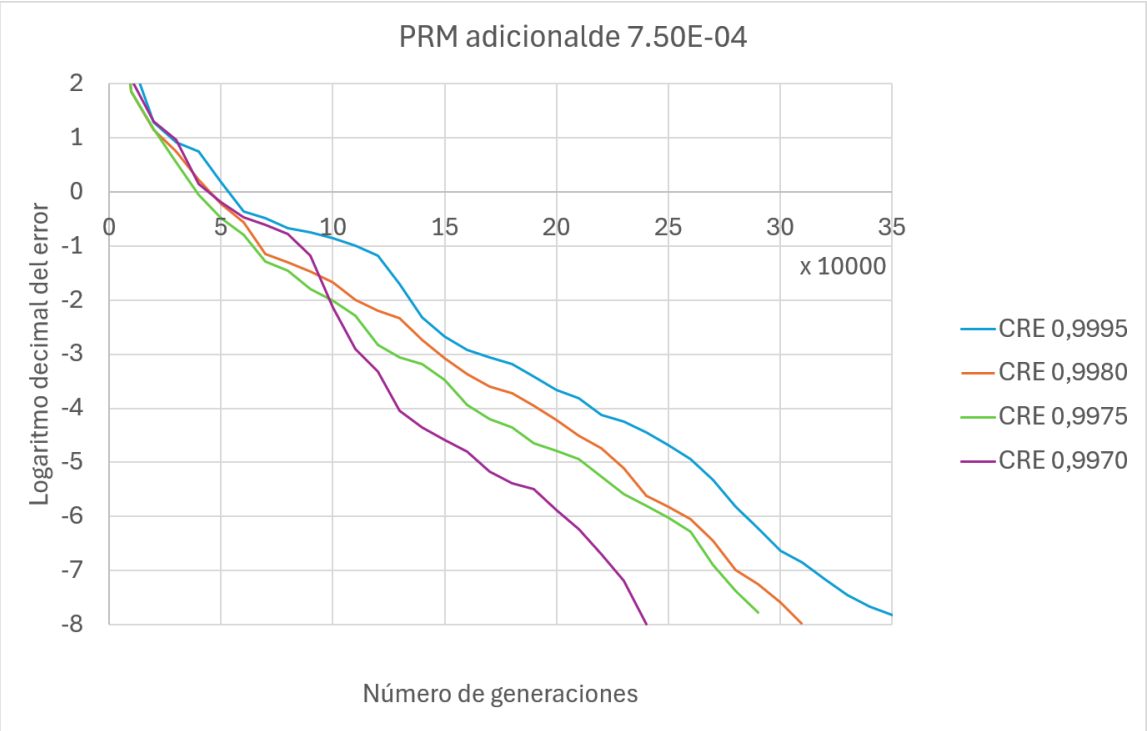


Figura 6. Evolución de los cálculos con el modelo de 63 sectores y una PRM adicional de 7,5E-03 (8.000E-04 de PRM total) cuando se usan como constante de recombinación los valores 0,9995 y 0,9975

Generar una cierta (aunque sea mínima) proporción de mutantes, cuasi iguales al mejor individuo de la generación anterior, acelera la convergencia del algoritmo hasta reducir en dos, o tres veces (incluso más), el número de generaciones necesarias para alcanzar el objetivo de error. El coste en tiempo de cálculo para retener el mejor individuo de la generación anterior es despreciable.

El precio de usar esta estrategia es que se aumenta la probabilidad de que la búsqueda de la solución queda atrapada en un mínimo local, cosa que ocurre cuando la PRM supera el 1%, en el caso de 31 sectores, o cuando supera el 0,15%, en el modelo de 63 sectores. Para un modelo determinado, cuanto mayor es el número de la población, mayor es el valor de la PRM a partir de la cual el cálculo no converge.

Inercia o pseudo gradientes.

En el anexo II del libro se muestro que el algoritmo “DE/best/1/bin” es en realidad una “caminata” aleatoria de paso variable por el espacio de las soluciones. Los mutantes son la forma de obtener los puntos a los que se desplazará el “caminante” y el mejor individuo no es más que el mejor punto encontrado (el punto de llegada) tras cada generación (el de mejor mérito) y que es el punto de partida para la generación siguiente.

Al terminar una generación, el “caminante” se ha desplazado del punto de partida al punto de llegada y emulando el comportamiento humano lo primero sería ver que ocurre si nos movemos un poco más allá del punto de llegada, siguiendo la misma dirección que traíamos (movimiento inercial), o retrocediendo un poco, por si nos hemos pasado de frenada. Calculamos el mérito del nuevo punto de llegada y si es mejor llevamos al “caminante” a ese punto, antes de iniciar la generación siguiente, tratando de acelerar la búsqueda del punto solución.

Esto es suponer que la dirección seguida por el “caminante”, desde el punto inicial hasta el punto final de una generación, es una especie de gradiente que indica hacia donde se mejora la solución. Como esta suposición es una simple conjetura, hay que contrastarla mediante los cálculos correspondientes, incorporando al algoritmo de búsqueda esta estrategia.

Los resultados muestran que esta estrategia no funciona, precisamente porque acelera la búsqueda de un mínimo que, la mayoría de las veces es un mínimo local. Opera de forma contraria a como lo hace la estrategia de individuos cuasi iguales al mejor.

Como ejemplo, en la tabla 4, se muestra la evolución de los cálculos de un escenario para el modelo de 31 sectores y 2 hogares con la estrategia de pseudo gradientes y en la tabla 5 ese mismo escenario sin dicha estrategia.

generación	d.est.poblac	ECT	VAB	impuestos	cotización SS	suma Z	suma Q	segs.p	paso+	paso-	med.mejora	max.mejora
1	3.50E+01	1.3439E+09	8.7730E+05	2.4638E+05	1.3656E+05	1.9573E+06	2.1834E+06					
10000	1.93E+00	3.4978E+04	8.1237E+05	2.2687E+05	1.2473E+05	1.8077E+06	2.0279E+06	67	8399	4916	2.16E+03	2.23E+06
20000	1.82E-01	4.7229E+03	8.0840E+05	2.2568E+05	1.2400E+05	1.7978E+06	2.0173E+06	70	5994	4962	6.16E-02	2.55E+00
30000	5.52E-03	2.9783E+03	8.0630E+05	2.2504E+05	1.2361E+05	1.7923E+06	2.0115E+06	70	4503	4853	4.27E-03	1.04E-01
40000	7.69E-03	2.9712E+03	8.0625E+05	2.2503E+05	1.2360E+05	1.7922E+06	2.0114E+06	70	3133	4891	2.14E-05	3.23E-04
50000	3.26E-02	7.7625E+02	8.0622E+05	2.2502E+05	1.2359E+05	1.7922E+06	2.0114E+06	71	3971	4772	4.41E-03	7.42E-01
60000	2.36E-02	3.0933E+02	8.0612E+05	2.2499E+05	1.2358E+05	1.7920E+06	2.0112E+06	72	3376	5147	1.36E-03	1.26E-01
70000	1.89E-03	2.9441E+02	8.0592E+05	2.2493E+05	1.2354E+05	1.7914E+06	2.0106E+06	72	3220	4681	4.53E-05	4.91E-04
80000	2.78E-04	2.9415E+02	8.0591E+05	2.2493E+05	1.2354E+05	1.7914E+06	2.0106E+06	71	558	3982	1.23E-06	1.40E-04
90000	1.24E-04	2.9412E+02	8.0591E+05	2.2493E+05	1.2354E+05	1.7914E+06	2.0106E+06	72	152	3853	1.72E-07	5.96E-06
Reinicio.						total segundos		**632**				
90000	4.30E+01	1.4017E+09	8.7828E+05	2.4665E+05	1.3678E+05	1.9613E+06	2.1871E+06	0				
100000	2.12E+00	1.0361E+05	8.2094E+05	2.2945E+05	1.2632E+05	1.8295E+06	2.0509E+06	62	8735	4670	1.74E+03	6.10E+05
110000	1.08E-01	3.4249E+03	8.0742E+05	2.2538E+05	1.2382E+05	1.7952E+06	2.0146E+06	69	7217	4710	2.18E-01	1.01E+01
120000	2.79E-02	1.4866E+03	8.0655E+05	2.2512E+05	1.2365E+05	1.7929E+06	2.0122E+06	69	4360	4745	5.39E-03	6.18E-01
130000	1.30E-03	1.4694E+03	8.0641E+05	2.2508E+05	1.2362E+05	1.7926E+06	2.0118E+06	72	3244	4625	5.68E-05	2.59E-03
140000	1.15E-02	9.2158E+01	8.0636E+05	2.2506E+05	1.2362E+05	1.7925E+06	2.0118E+06	70	3846	4649	3.82E-03	4.60E-01
150000	9.76E-04	1.2259E+00	8.0623E+05	2.2502E+05	1.2359E+05	1.7922E+06	2.0114E+06	72	3338	4577	3.18E-04	4.71E-02
160000	1.17E-03	2.2268E-02	8.0622E+05	2.2502E+05	1.2359E+05	1.7922E+06	2.0114E+06	74	1098	4244	5.44E-06	8.16E-04
170000	5.53E-05	4.7269E-05	8.0621E+05	2.2502E+05	1.2359E+05	1.7921E+06	2.0114E+06	74	86	3192	1.68E-07	2.23E-06
180000	1.65E-06	8.1579E-08	8.0621E+05	2.2502E+05	1.2359E+05	1.7921E+06	2.0114E+06	73	0	620	1.07E-09	4.35E-09
190000	4.62E-07	6.6645E-10	8.0621E+05	2.2502E+05	1.2359E+05	1.7921E+06	2.0114E+06	72	0	0		
198691	4.52E-08	9.9947E-12	8.0621E+05	2.2502E+05	1.2359E+05	1.7921E+06	2.0114E+06	63	23.4			
						total segundos		**1340**				

Tabla 4. Resultados de la estrategia de pseudo gradientes.

generación	d.est.poblac	ECT	VAB	impuestos	cotización SS	suma Z	suma Q	segs.p
1	3.53E+01	1.3461E+09	8.7735E+05	2.4639E+05	1.3657E+05	1.9575E+06	2.1836E+06	
10000	4.33E-01	2.3487E+04	8.1185E+05	2.2671E+05	1.2462E+05	1.8066E+06	2.0267E+06	68
20000	1.22E-01	5.5186E+02	8.0719E+05	2.2531E+05	1.2377E+05	1.7946E+06	2.0140E+06	74
30000	6.94E-03	7.7879E+00	8.0632E+05	2.2505E+05	1.2361E+05	1.7924E+06	2.0116E+06	78
40000	1.70E-03	9.1376E-02	8.0622E+05	2.2502E+05	1.2359E+05	1.7922E+06	2.0114E+06	76
50000	1.33E-04	4.4999E-04	8.0621E+05	2.2502E+05	1.2359E+05	1.7921E+06	2.0114E+06	76
60000	7.46E-06	3.7144E-06	8.0621E+05	2.2502E+05	1.2359E+05	1.7921E+06	2.0114E+06	76
70000	1.05E-06	1.0837E-08	8.0621E+05	2.2502E+05	1.2359E+05	1.7921E+06	2.0114E+06	74
80000	5.17E-08	6.7455E-11	8.0621E+05	2.2502E+05	1.2359E+05	1.7921E+06	2.0114E+06	72
83500	2.60E-08	9.9982E-12	8.0621E+05	2.2502E+05	1.2359E+05	1.7921E+06	2.0114E+06	25
							total segundos	**619**

Tabla 5. Resultados sin la estrategia de pseudo gradientes.

paso+ y paso- son el número de desplazamientos (por inercia) hacía adelante (avances) y hacia detrás (retrocesos) del punto de llegada, en cada bloque de 10000 generaciones. En las dos columnas siguientes: la mejora media del mérito con estos desplazamientos inerciales y la máxima en una generación.

El número de paso+ es inicialmente mayor que el de paso- porque la aplicación siempre prueba, en primer lugar, a moverse hacia adelante, y solo si esto no mejora el mérito, prueba a retroceder. Pero según nos acercamos a la solución los retrocesos les van ganado terreno a los avances, lo cual es lógico ya que la longitud de los pasos que da el “caminante” se va reduciendo según nos acercamos a la solución, como se muestra en el Anexo II del libro. Si la longitud de un paso en una generación es mayor que la que tendrá en la siguiente y es la que aplicamos al terminar la generación antes de empezar la siguiente, es lógico que nos pasemos de frenada.

Se trata de un ejemplo porque el carácter estadístico del algoritmo puede conducir a evoluciones distintas en función del generador de números aleatorios, pero a la vista de los casos que hemos analizado, en general, se cumple que la estrategia con pseudo gradientes es menos eficiente que sin ella.

El espacio de soluciones del modelo de 31 sectores y 2 hogares tiene 251 dimensiones. Como solo somos capaces de visualizar un máximo de 3 dimensiones, para hacernos una idea de que mérito corresponde a cada punto, hay que acudir a la reducción de dimensiones. El método clásico es el análisis de componentes principales (PCA) pero para relaciones no lineales es preferible el t-SNE (t-Distributed Stochastic Neighbor Embedding) del que solo diremos que calcula la probabilidad de que dos puntos sean vecinos utilizando una distribución gaussiana centrada en cada punto.

La reducción de dimensiones no está relacionada con el mérito de cada punto, lo que hace es buscar un nuevo sistema de referencia (coordenadas) de forma que en este sistema los puntos aparezcan lo más separados posibles los unos de los otros. El mérito lo usamos a posteriori, representamos los puntos en el nuevo sistema de coordenadas y le asignamos un color en función del mérito de cada uno para ver si los puntos con mérito similar ocupan posiciones cercanas, o dicho de otra forma, para ver si los puntos cercanos (coordenadas parecidas) tienen méritos parecidos.

Generamos 2000 puntos al azar muy próximos al punto de equilibrio (el de la MCS) del modelo de 31 sectores y 2 hogares. Mediante PCA y t-SNE los reducimos a 2 dimensiones y los representamos en los gráficos de las figuras 7 y 8. Cada uno de los ejes corresponde a los valores de una combinación (una para cada eje) de las 251 variables del espacio original. Estas combinaciones o componentes principales son el resultado de los métodos de reducción y son, como hemos dicho, las que mejor separan los puntos entre sí, es decir, las que maximizan las distancias entre los puntos en el espacio de 2 dimensiones.

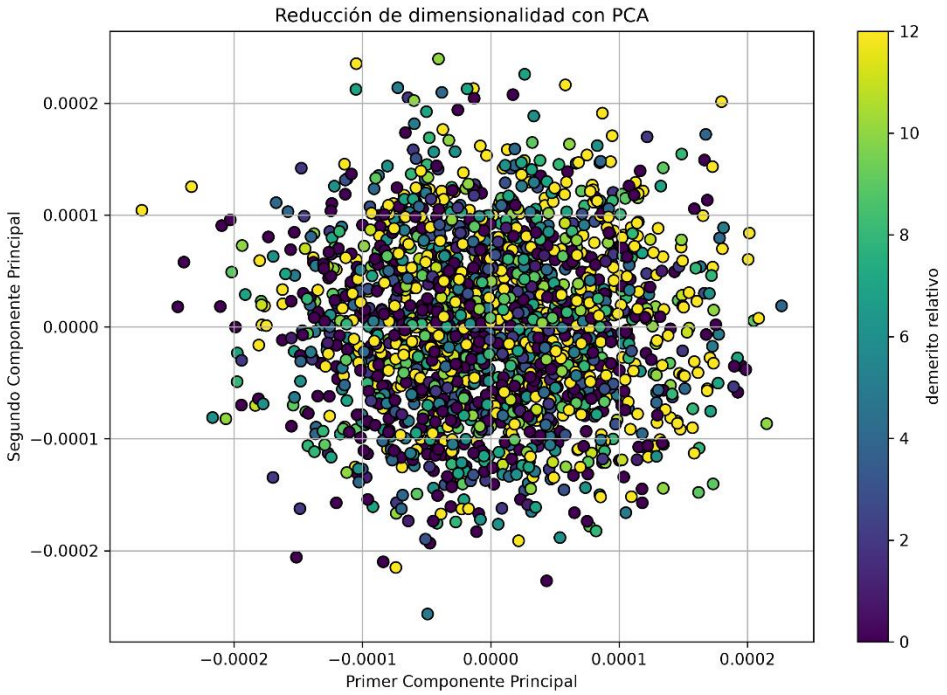


Figura 7. Reducción a 2 dimensiones, mediante PCA, del espacio de 251 dimensiones del modelo de 31 sectores y 2 hogares.

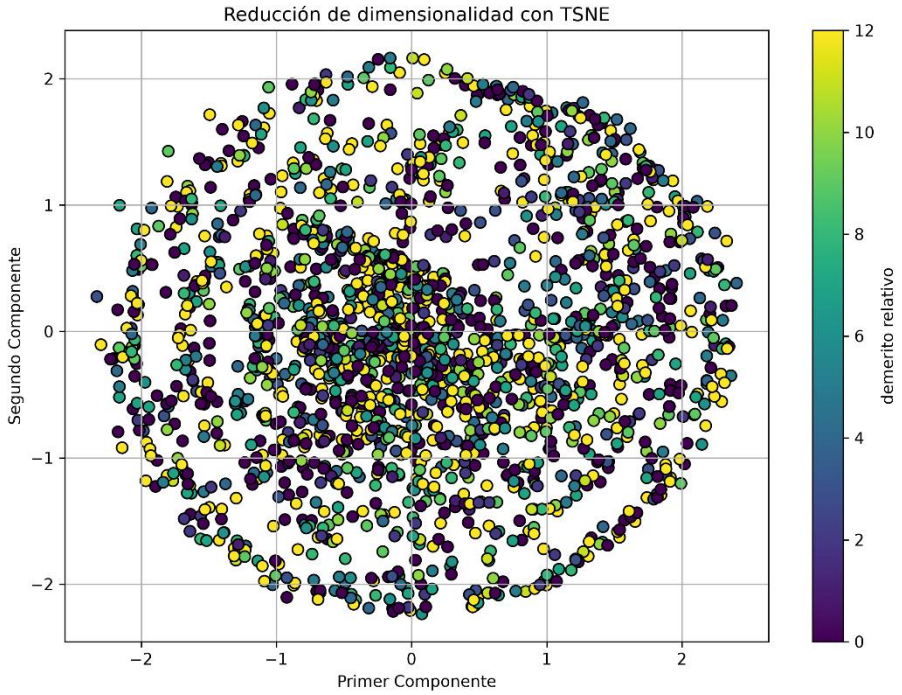


Figura 7. Reducción a 2 dimensiones, mediante t-SNE, del espacio de 251 dimensiones del modelo de 31 sectores y 2 hogares.

Lo primero que se observa es que el t-SNE es mucho más eficiente separando los puntos que el PCA, pero lo que es relevante es que no se consigue separar, ni siquiera mínimamente, los grupos de puntos con un mérito similar. Encontramos a los puntos con el mayor mérito (azul oscuro) entremezclados con los de menor mérito (amarillo) por toda la superficie del gráfico. Esto confirma que no existen gradientes en nuestra función de mérito y muestra por qué la estrategia de seudo gradientes no funciona.

La métrica del espacio de soluciones es incomparablemente más compleja que la geometría de nuestro mundo (incluso considerando la relatividad) y por eso no funciona el intento de trasladar al algoritmo de búsqueda nuestra forma de pensar inercial dentro de una geometría euclidiana.

Abundando en la complejidad de nuestra función de mérito, he tratado de "aproximarla" mediante una red neuronal, en versiones densa y convolucional con resultados concluyentes: no es posible.

Para el modelo de 31 sectores y 2 hogares, he preparado una muestra con 20000 "individuos" generados al azar con valores muy próximos a la solución de equilibrio, las diferencia para cada variable, con respecto a su valor de equilibrio, es del orden de 10^{-5}, podríamos decir que son diferencias infinitesimales y sin embargo el mérito, calculado por nuestra función, va desde 10^{-10} hasta 10^{+05}. Aunque hay una cierta concentración de valores en una franja de mérito cercano a cero, hay una amplia distribución por todo el rango de méritos mencionado (figura 8).

Los 20000 individuos se fraccionan en tres muestras con 14000, 4000 y 2000 individuos respectivamente. La primera sirve para entre la red, la segunda es la de validación y la tercera de prueba. Ambos tipos de redes neuronales llegan a la misma solución: la función que "aproxima" el mérito es un valor constante igual para todos los individuos, igual al valor medio del mérito de todos los individuos de la muestra, como puede verse en la figura 9.

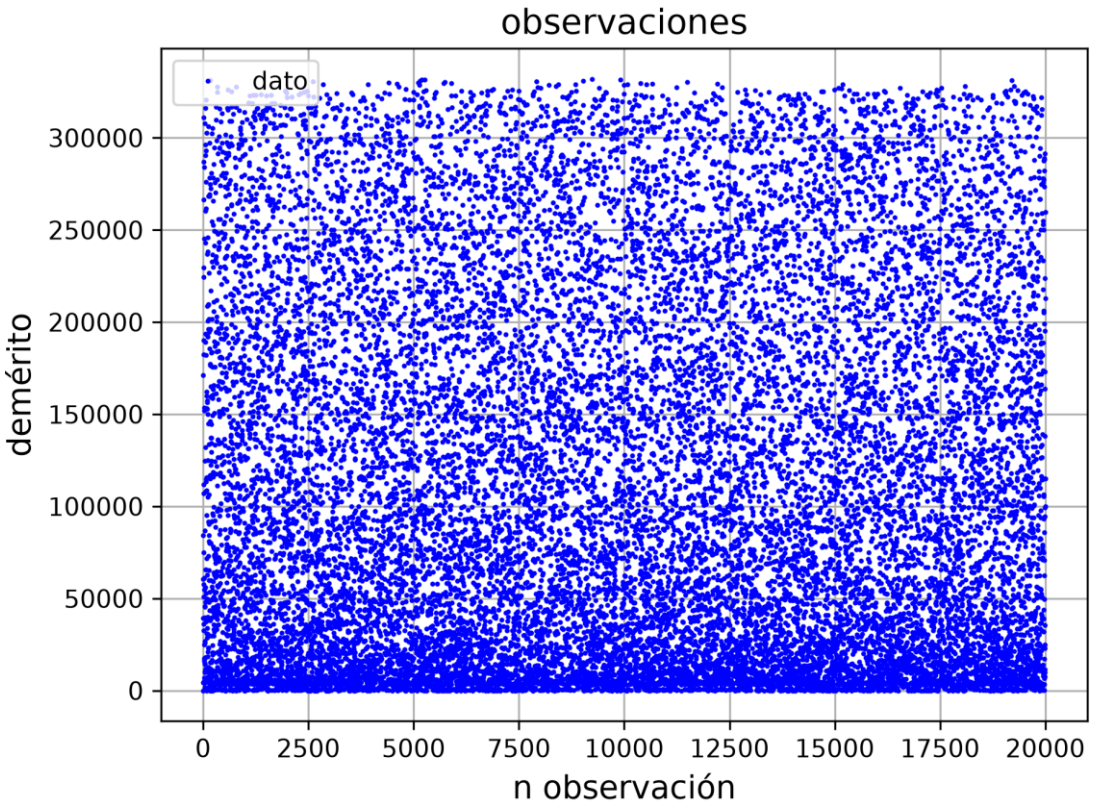


Figura 8. Demérito de los 20000 individuos generados en el entorno de la solución de equilibrio del modelo de 31 sectores y 2 hogares.

Las redes neuronales no detectan ninguna funcionalidad (patrón) aunque nuestra función está perfectamente definida y para cualquier vector de valores de las variables (individuo) calcula valor del mérito que le corresponde.

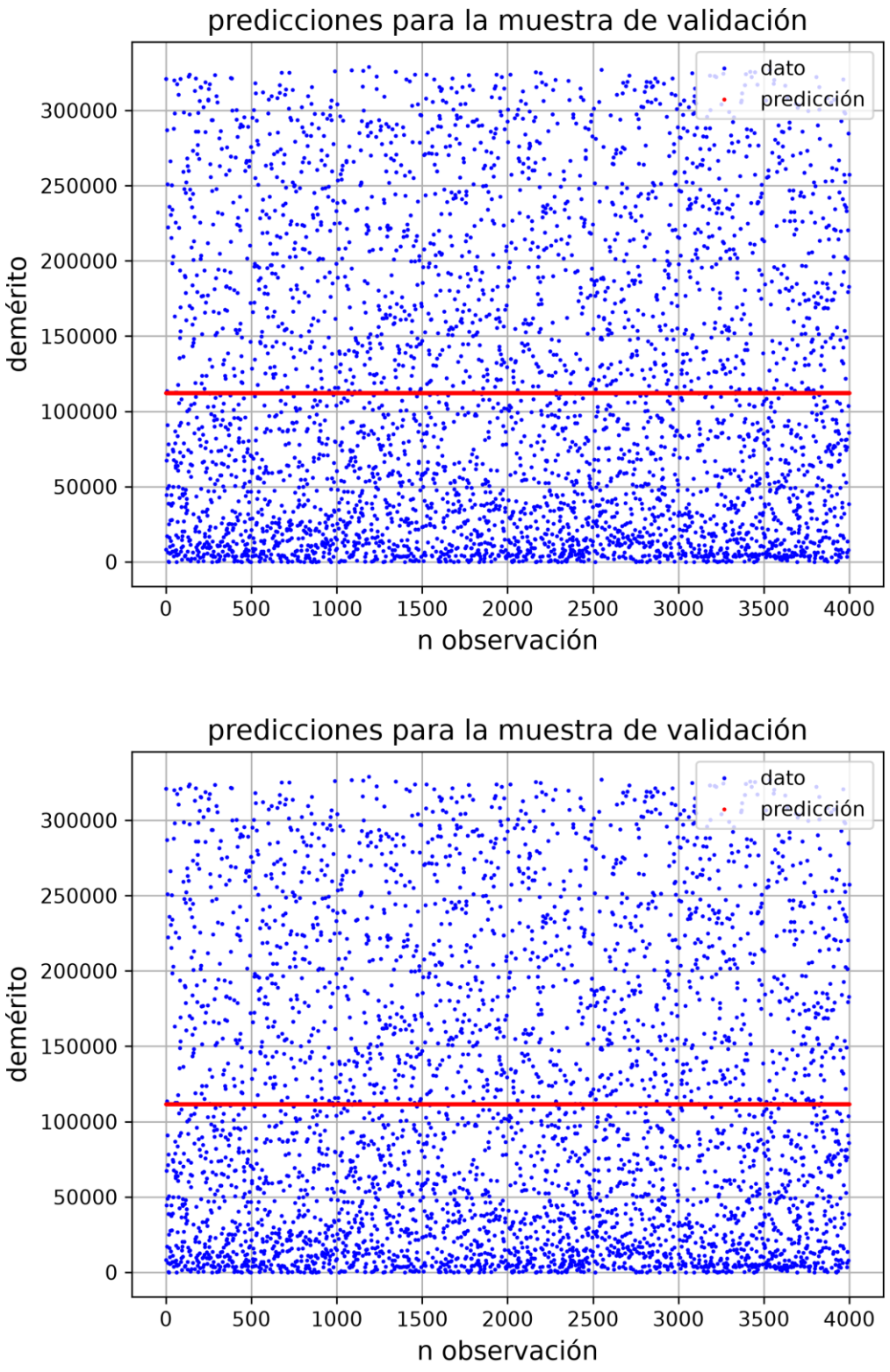


Figura 9. Representación de los valores del mérito (demérito) para los 4000 individuos (n observación) de la muestra de validación y de la predicción correspondiente a cada uno de ellos. Arriba con la red neuronal convolucional y abajo con la densa.

Utilizar la GPU (unidad de procesamiento gráfico).

El algoritmo genético es fácilmente paralelizable, cosa que ya hemos aprovechado de forma decisiva, pero queda por explorar la ejecución del modelo en una GPU en lugar de la CPU.

Los modelos de IA son el prototipo de cálculos paralelizables y por eso se han desarrollado extraordinariamente las unidades de procesamiento gráfico que, aunque se crearon para conectar un ordenador personal a un monitor, se vio que podían ser muy útiles para acelerar el entrenamiento de los modelos de IA y de simulación técnico-científicos, porque su arquitectura estaba pensada para llevar de forma simultánea (en paralelo) los datos de la memoria a los pixeles del monitor.

En las especificaciones de las GPU actuales se mencionan miles, incluso, decenas de miles de núcleos CUDA[4] y centenares de miles de hilos (thread), pero estos no son comparables a los núcleos o hilos de una CPU, son más bien una organización administrativa, una forma de organizar (fragmentar) en "tareas" los cálculos y planificar su ejecución. Estas tareas (procesos) no se ejecutan de forma simultánea, lo hacen conforme a unas reglas relacionadas con un gran número de factores entre los que destacan los Multiprocesador Streaming (SM) que se pueden comparar con los núcleos de una CPU (aunque menos versátiles), cada SM es capaz de ejecutar un conjunto de "bloques" y cada uno de estos bloques administra un determinado número de hilos, de los cuales solo un reducido número de estos (típicamente 32) se ejecuta de forma simultánea (warp). Además, el número de tareas que se procesan en paralelo está limitado, entre otros condicionantes, por la memoria disponible y las necesidades de memoria de los cálculos a realizar en cada hilo.

Al final, el número de procesos en paralelo, que ejecuta una GPU, es muy inferior a lo que sugieren los miles de núcleos CUDA, aunque si mayor que lo que puede procesar una CPU.

Una ventaja importante de las GPU es que se pueden usar varias de forma conjunta, escalando la capacidad de proceso sin tener que modificar las aplicaciones, aunque el coste de los equipos se escala en la misma medida.

Nuestras pruebas se han realizado con una sola GPU de las llamadas de "consumo" destinadas al público en general, las GPU más potentes tienen un coste de decenas de miles de euros, prohibitivo para uso personal.

Nuestra GPU de consumo es una NVIDIA Quadro RTX 6000 con 24 Gb de memoria RAM, destacada hace 10 años, pero hoy superada por muchas otras.

Para los cálculos he modificado la librería DLL para convertirla en una aplicación de escritorio C++/CUDA. La DLL recibe los datos del caso a calcular

[4] CUDA es el acrónimo de "Compute Unified Device Architecture" que es una plataforma de computación en paralelo y un modelo de programación creado por NVIDIA.

directamente de la aplicación Python (que es quien la llama) y la nueva aplicación (MEGA_GPU) los lee de un fichero de texto que se genera con la aplicación Python, no hay conexión directa entre ambas aplicaciones.

Los resultados son que la aplicación que usa la GPU es aproximadamente 1,5 veces más rápida que la versión CPU.

El aumento de velocidad es mucho menor de lo esperado, a la vista de las capacidades de la GPU. Esto es así, fundamentalmente, porque la función (merito) que deben ejecutar los hilos es muy compleja (todas las ecuaciones del modelo) y requiere mucha memoria, el doble de todas las variables del modelo, para los dos pasos que calcula la función objetivo.

Ya hemos dicho que los hilos asignados a un bloque se agrupan en “paquetes” (warp) de 32 hilos que se ejecutan simultáneamente (en paralelo) para la misma función con diferentes datos. Por su parte, los bloques se asignan a los Multiprocesador Streaming (SM) pero solo un determinado número de bloques están activos simultáneamente.

Un dato crítico es que existe un número máximo de registros por SM (65536) que se comparten por los bloques asignados al mismo y se usan para almacenar las variables locales de cada hilo, esto limita el número de bloques que pueden estar activos simultáneamente en una SM y por tanto el de warp, lo que en definitiva determina el número de hilos que se ejecutan en paralelo (lo que se denomina ocupación de la GPU). Además, cuando los hilos necesitan más capacidad de almacenamiento que el número de registros de los que disponen, deben recurrir a la memoria global de la GPU (lo que se llama spilling, desbordamiento o derrame). Los registros son una parte de la memoria de la GPU a la que se accede muy rápido y la diferencia con la memoria global es muy grande, mientras para acceder a un registro se consume 1 ciclo de reloj, el acceso a la memoria local puede necesitar cientos.

Esto nos lleva a tener que elegir entre reducir el número de registros por hilo para maximizar la ocupación (el máximo número de hilos en activo), o aumentar el número de registros para minimizar el spilling, aunque se reduzca la ocupación. Elección que hay que hacer mediante prueba y error porque no tenemos una función de utilidad de la que podamos echar mano. Para el modelo de 63 sectores y 3 hogares hemos encontrado los mejores resultados si se eligen 72 bloques y 256 hilos lo que permite que cada hilo procese un solo individuo si la población es de 18432 individuos, un número perfectamente válido según lo que hemos visto en el cálculo de escenarios que se analizan en el libro.

La función crítica de nuestro modelo es la que realiza la “selección” de los nuevos individuos de la población porque requiere calcular la función de mérito para todos y cada uno de los mutantes.

Para el modelo de 63 sectores y 3 hogares esta función requiere almacenar 101776 bytes (cumulative stack). Si se usan 255 registros (el máximo) por hilo[5], cada hilo tiene que recurrir a la memoria global para escribir 21928 bytes y leer 34476 bytes. CUDA solo puede asignar un bloque por SM y solo 8 warps por SM.

Si se restringe el número de registros por hilo a 64, CUDA puede asignar 4 bloques por SM y 32 warps por SM (aumenta la ocupación), pero el spilling (aumenta) sube a escribir 39544 bytes y leer 67800 bytes y el resultado conjunto es que aumenta ligeramente el tiempo de cálculo.

En definitiva, nuestro modelo trabaja con una ocupación muy baja y un derrame (spilling) muy alto lo que conduce a unos pobres resultados, sin que esto se pueda mejorar manipulando las opciones del compilador CUDA.

Para los cálculos relacionados con la IA se necesitan números en coma flotante pero no de gran precisión, lo normal es usar el tipo float simple de 32 bits (FP32) que ofrece unas 8 cifras significativas, e incluso de menor precisión como FP8 y hasta FP4. Sin embargo, para los cálculos científicos y el MEGA hay que encuadrarlo aquí, se necesita, al menos, doble presión (FP64 con 15 cifras significativas). Las GPU actuales pueden operar con FP32 y FP64, pero están orientadas al FP32 de forma que los cálculos con FP64 consumen aproximadamente 16 veces más de tiempo que con FP32. Existen tarjetas donde los cálculos FP64 son sólo 2 veces más lentos que los FP32, pero estas tarjetas tienen precios que no son asequibles para uso personal. Si se ejecuta el MEGA en presión simple (FP32) el modelo calcula 3 veces más rápido (no 16 veces, porque no todo son operaciones entre números) pero colapsa rápidamente muy lejos de una solución aceptable.

La mayor fabricante de GPU, NVIDIA, además de las tarjetas ofrece un servicio (NEBIUS) de alquiler de tiempo de cálculo en servidores equipados con un gran número de las mejores GPU para ejecutar proyectos muy exigentes.

[5] Hilos por SM = 65536 / 255 (registros por hilo) = 257

Warps por SM = 257 / 32 = 8 warps activos.